MARQ: A Multi-Agent Rainbow Deep Q-Networks Framework using AAREN for Mastering Imperfect Information Games

James Gabriel P. Mabagos
Bachelor of Science in Computer Science

Mark Lawrence M. Quibot
Bachelor of Science in Computer Science

Senior Thesis submitted to the faculty of the
Department of Computer Science
College of Computer Studies, Ateneo de Naga University
in partial fulfillment of the requirements for their respective
Bachelor of Science degrees

Project Advisor: Kevin G. Vega
Estrella H. Montealegre
Michelle Elija B. Santos
Ian Peter L. Lastimosa

September 5 2025
Naga City, Philippines

Keywords: Imperfect Information, DQN, Multi-Agent

The Senior Project entitled

MARQ: A Multi-Agent Rainbow Deep Q-Networks Framework using AAREN for Mastering Imperfect Information Games

developed by

James Gabriel P. Mabagos

Bachelor of Science in Computer Science

Mark Lawrence M. Quibot

Bachelor of Science in Computer Science

and submitted in partial fulfillment of the requirements of their respective Bachelor of Science degrees has been rigorously examined and recommended for approval and acceptance.

Estrella H. Montealegre
Panel Member
Date signed: _____

Michelle Elija B. Santos
Panel Member
Date signed: _____

Ian Peter L. Lastimosa
Panel Member
Date signed: _____

Kevin G. Vega
Project Advisor
Date signed: _____

The Senior Project entitled

MARQ: A Multi-Agent Rainbow Deep Q-Networks Framework using AAREN for Mastering Imperfect Information Games

developed by

James Gabriel P. Mabagos
Bachelor of Science in Computer Science

Mark Lawrence M. Quibot
Bachelor of Science in Computer Science

and submitted in partial fulfillment of the requirements of their respective Bachelor of Science degrees is hereby approved and accepted by the Department of Computer Science, College of Computer Studies, Ateneo de Naga University.

Lowie Vincent S. Bisana MIT
Chair, Department of Computer Science
Date signed: _____

Joshua C. Martinez MIT
Dean, College of Computer Studies
Date signed: _____

Declaration of Original Work

We declare that the Senior Project entitled

MARQ: A Multi-Agent Rainbow Deep Q-Networks Framework using AAREN for Mastering Imperfect Information Games

which we submitted to the faculty of the

Department of Computer Science, Ateneo de Naga University

is our own work. To the best of our knowledge, it does not contain materials published or written by another person, except where due citation and acknowledgement is made in our senior project documentation. The contributions of other people whom we worked with to complete this senior project are explicitly cited and acknowledged in our senior project documentation.

We also declare that the intellectual content of this senior project is the product of our own work. We conceptualized, designed, encoded, and debugged the source code of the core programs in our senior project. The source code of third party APIs and library functions used in our program are explicitly cited and acknowledged in our senior project documentation. Also duly acknowledged are the assistance of others in minor details of editing and reproduction of the documentation.

In our honor, we declare that we did not pass off as our own the work done by another person. We are the only persons who encoded the source code of our software. We understand that we may get a failing mark if the source code of our program is in fact the work of another person.

James Gabriel P. Mabagos
4 - Bachelor of Science in Computer Science

Mark Lawrence M. Quibot
4 - Bachelor of Science in Computer Science

This declaration is witnessed by:

Kevin G. Vega
Project Advisor

MARQ: A Multi-Agent Rainbow Deep Q-Networks Framework using AAREN for Mastering Imperfect Information Games

by

James Gabriel P. Mabagos and Mark Lawrence M. Quibot

Project Advisor: Kevin G. Vega

Department of Computer Science

ABSTRACT

This research presents MARQ, a Multi-Agent Rainbow Deep Q-Networks, designed to master imperfect information games through the board game Stratego. MARQ leverages a Deep Recurrent Q-Network (DRQN) architecture and league-based self-play training to address the large state space of Stratego and hidden information challenges. The architecture integrates an Attention as Recurrent Neural Network (AAREN) to maintain refined belief states over long game histories, enabling opponent modeling and strategic value contextualization. The system employs Distributional Reinforcement Learning with Noisy Networks to balance strategic unpredictability and stability. The methodology involves developing a game environment and training the AI agent through multi-channel state representations, incorporating Probabilistic Belief States (PBS) and weighted piece values computed through systematic power analysis to optimize decision making under uncertainty. A structured participant study introduces enthusiasts without prior Stratego experience to create a synchronized learning environment, combining quantitative metrics including win rate percentage, average game length, and decision-making speed, with qualitative usability assessments across performance, reliability, and applicability dimensions. Through this study MARQ aims to deliver strategic decisions comparable to human opponents.

ACKNOWLEDGEMENTS

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

Chapter 1

Introduction

1.1 Project Context

Games have a history of being a metric for how Artificial Intelligence (AI) progresses and performs in the current time [32]. Many real-world problems, from military tactics to multiplayer games, require intelligent decision-making in environments where all information is not available. A common example is a feature found in many multiplayer games known as the "fog of war" [39], where the game environment is not fully revealed, which makes it an example of an imperfect information game [21]. Imperfect information games create a scenario where players develop unconventional strategies without having full knowledge of the game's environmental state [39]. Solving these kinds of problems and determining the optimal strategy remains a significant challenge for Artificial Intelligence [34].

This study focuses on mastering imperfect information games through Multi-Agent Reinforcement Learning (MARL) with Deep Q-Networks (DQN). In MARL, multiple agents learn and adapt in the same environment, making each decision based on its own observations and rewards [11, 8]. With DQN, the agents can utilize deep neural networks to approximate the Q-function that maps state-action pairs for the expected reward. With all of these together, this study introduces MARQ (Multi-Agent Rainbow Deep Q-Networks), a framework to solve imperfect information games. This approach is suited well for Stratego, where these agents are trained to compete against each other, developing strategies through repeated experience, even though information is lacking about the opponent's setup or the environment's full state.

1.2 Purpose and Description

This research aims to evaluate whether Multi-Agent Reinforcement Learning (MARL) with Deep Q-Networks (DQN) can be used to develop intelligent agents capable of mastering imperfect information games. By focusing on the limited information available to players in environments such as the board game Stratego, this study seeks to address the challenge of making decisions under uncertainty while incorporating strategic planning. Although existing tools for Stratego provide some analytical support, they are often limited in fostering deeper reasoning about the game's tactical complexities.

To advance beyond these limitations, this research seeks to provide an algorithmic foundation for future analytical systems while also emphasizing the development of critical thinking in solving imperfect information games. By modeling adaptive strategies and reasoning under uncertainty, the study highlights how artificial intelligence can mirror and enhance human-like logical reasoning, offering valuable insights for both game analysis and broader decision-making domains.

1.2.1 Research Questions

This study is guided by three main research questions:

1. How effective is the AI agent in adapting to hidden information and different strategies?
2. To what extent does the trained AI agent's performance compare against human players and other AI opponents in Stratego?
3. How does the use of Stratego as a research platform contribute to advancing studies in imperfect information multi-agent reinforcement learning?

1.3 Objectives

The main objective of this study is to develop and evaluate Multi-Agent Reinforcement Learning with Deep Q-Networks for learning appropriate strategies in an imperfect information game, where the agents operate under hidden state variables to improve decision-making, adaptability, and strategic reasoning against both human and AI opponents. This can be achieved through the following:

- Introduction of Stratego to the community

- Design and implement a game environment for Stratego
- Build and implement a training model using Multi-Agent Reinforcement Learning with Deep Q-Networks and AAREN.
- Train and optimize the AI agent to respond to varying game states, hidden information, and opposing strategies.

1.4 Scope and Limitation

The study is limited to the development of an AI agent for imperfect information games, Stratego being the test environment. The scope includes the design and implementation of Stratego as a game environment, the development of a training framework using Multi-Agent Reinforcement Learning (MARL) with Deep Q-Networks (DQN) using AAREN, and the training and optimization of the AI agent to adapt to differences in game states. The evaluation of the agent's performance would primarily be on Stratego's environment. The trainings are limited to the personal devices of the researchers and would not rely on cloud-based computing resources. Since the AI is going to learn alongside the community the player and AI head-to-head will be analyzed mostly qualitative. The goal will be achieving the same human critical thinking in the model.

1.5 Significance of the Study

The development of MARQ addresses limitations in current imperfect information game AI, where existing systems either require large computational resources or employ model-free approaches that limit strategic depth. MARQ's significance lies in its explicit opponent modeling through AAREN and probabilistic belief states, and hybrid DQN architecture that balances strategic depth with computational tractability. The synchronized learning evaluation framework provides insights into AI adaptability against improving human opponents, with implications extending beyond gaming to domains characterized by incomplete information and strategic interaction, including cybersecurity, financial trading, and strategic planning.

1.6 Definition of Terms

AAREN

Attention a recurrent neural network, a model that integrates the sequential processing of RNNs with the parallel efficiency of Transformers, designed for tasks such as event forecasting and classification.

Alpha-Beta Pruning

A search algorithm that explores possible moves in a game tree while pruning branches that cannot affect the final decision, making the search process more efficient.

Counterfactual Regret Minimization (CFR)

An iterative algorithm that approximates Nash equilibrium by minimizing regret, widely applied in imperfect information games such as Poker and Stratego.

Deep Q-Networks (DQN)

A reinforcement learning algorithm that combines Q-learning with deep neural networks, enabling agents to approximate optimal decision-making in complex environments.

Fog of War

A game mechanic used in strategy games that conceals parts of the map or opponent actions, requiring players to utilize prediction and information-gathering strategies to reduce uncertainty.

Imperfect Information Games

Games where players have incomplete knowledge of the current game state, requiring them to make strategic decisions under uncertainty (e.g., Stratego, Poker, Multiplayer Online Battle Arenas).

Knowledge-Limited Unfrozen Subgame Solving(KLUSS)

Knowledge-Limited Unfrozen Subgame Solving (KLUSS) is an advancement in search algorithms for imperfect-information games. KLUSS builds on the earlier Knowledge-Limited Subgame Solving (KLSS) method, which simplified computation by removing irrelevant states and freezing certain

strategies.

Long Short-Term Memory (LSTM)

A variant of RNN designed to capture long-term dependencies in sequential data while mitigating the vanishing gradient problem.

Multi-Agent Reinforcement Learning (MARL)

An extension of reinforcement learning in which multiple agents interact within the same environment, learning coordinated or competitive strategies through simultaneous training.

Nash Equilibrium

A game-theoretic concept where no player can gain an advantage by unilaterally changing their strategy, often used as a benchmark for decision-making in imperfect information games.

Perfect Information Games

Games in which all players have complete knowledge of the current game state and available moves, such as Chess and Go.

Proximal Policy Optimization (PPO)

A reinforcement learning algorithm that optimizes policies through probability ratio clipping, often compared to DQN in performance within gaming environments such as VizDoom.

Recurrent Neural Network (RNN)

A type of neural network designed to process sequential data by maintaining memory of previous inputs, useful in imperfect information games for recalling past moves or revealed information.

Reinforcement Learning (RL)

A machine learning paradigm where agents learn optimal strategies through trial-and-error interactions with the environment, guided by rewards and penalties.

Stratego

A two-player strategy board game characterized by hidden information and asymmetric piece abilities, where the objective is to capture the opponent's flag or render them immobile.

Student of Games (SoG)

A unified AI framework combining self-play learning, strategic search, and game-theoretic reasoning, designed to handle both perfect and imperfect information games.

Vanishing Gradient

The vanishing gradient problem is basically as in the name, vanishing gradient, as color in a gradient, the message in this case starts strong and gradually fades away when it passes through each layer, making the learning weak or close to zero.

Chapter 2

Review of Related Literature

2.1 Artificial Intelligence in Imperfect Information Games

Games can be classified as an imperfect information game by having incomplete information about the current game state for both players [32, 11]. An example would be in MOBAs (Multiplayer Online Battle Arena), the fog of war mechanic hides portions of the map, including objectives and enemy positions, from both teams. By placing a tool, such as a ward in League of Legends or an observer in Dota 2, the player gains a temporary vision in the map for strategic planning [9]. This feature engages players to create strategic reasoning under certainty, utilizing prediction, coordination, and resource management to gain informational advantages over opponents. Games have a history of being a metric on how AI progresses and performance in the current time [32, 15]. The case is the same for both perfect information and imperfect information games [32]. For perfect information games, like chess, they primarily use Alpha-Beta pruning, a search algorithm that uses trees to explore possible chess moves and prunes the branches that will not affect the final decision [26]. In imperfect information games, a search algorithm like Alpha-Beta Pruning would be difficult to implement because the algorithm is made for perfect information games [31]. A popular algorithm that is used in imperfect information games would be Monte Carlo Tree Search with Reinforcement Learning because MCTS has integration with neural network systems like AlphaZero [27]. In terms of reinforcement learning, the training focuses on trial and error to provide the best possible decision [42]. Continuing development of AI algorithms in imperfect information games

advances our understanding of decision-making under uncertainty and promotes strategic planning.

2.1.1 Student of Games

A unified learning algorithm that can support both perfect information and imperfect information is Student of Games. Combining self-play learning, strategic search, and principles from game theory [32]. SoG achieved strong performance in Chess and Go in perfect information games and in imperfect information games like heads-up no-limit Texas hold 'em poker, and defeated the state-of-the-art agent in Scotland Yard. When comparing to MARL, SoG supports and has been tested in both game types and uses game theoretic reasoning, which computes or approximates the Nash equilibria. MARL with DQN uses trial and error learning and self-play learning, which relies on the exploration of the environment. MARL has also been tested in Starcraft II, which is a real-time strategy video game, unlike SoG, which has not been tested in video games [18].

2.1.2 Deep Q-Networks

One of the most important deep-learning algorithms is called Deep Q-Networks [16]. This algorithm is a type of reinforcement learning algorithm that combines Q-Learning with deep neural networks [6]. A study has been made comparing DQN with Proximal Policy Optimization(PPO) in VizDoom, a 3D environment for reinforcement learning based on Doom, a first-person-based shooter [37]. In the study, the environment would be in a gamemode called DeathMatch, where the main goal is to kill as many bots while minimizing deaths. In PPO with object detection, it learned a policy to run around and collect survival items like medkits and armor rather than focusing on kills, making the bot have close to zero kills. While the bot using DQN with object detection would go for both kills and collecting survival items, making the kill-death ratio higher than PPO. In short, PPO with object detection focuses on survival by collecting survival items and avoiding combat, while DQN with object detection has a balance of collecting survival items and combat.

2.1.3 Multi-Agent Reinforcement Learning

Reinforcement Learning can solve complex problems through trial and error, by learning from the environment to be able to provide optimal decisions [11]. This can be the case for Single-Agent Reinforcement Learning(SARL). SARL has its limits in solving many real-world problems. Multi-

Agent Reinforcement Learning(MARL) was introduced as a solution for the challenges of SARL, by having multiple agents in an environment that interact with each other, the decisions they could make would be optimized [11, 8]. Optimized, meaning that these agents go through trial and error together to make the best decision given the reward. As for the implementation of this in Stratego, two agents would compete with each other to know the optimal decisions given the environment, which has imperfect information and opposing strategies. As MARL has its advantages, it also has challenges. The first would be Non-Stationarity, and the second is scalability, as for non-stationarity, each agent learns simultaneously, meaning the environment constantly changes [8]. For scalability, as more agents are being used, the computational power rises, making it expensive [23, 8].

2.1.4 KLUSS

Knowledge-Limited Unfrozen Subgame Solving (KLUSS) was introduced as an advancement in search algorithms for imperfect-information games.[39] KLUSS builds on the earlier Knowledge-Limited Subgame Solving (KLSS) method, which simplified computation by removing irrelevant states and freezing certain strategies. Unlike KLSS, KLUSS "unfreezes" these strategies, allowing them to be re-optimized during subgame solving. This flexibility improves strategic reasoning, such as bluffing and adaptation, and was a key factor in achieving high-level performance in Fog of War chess.

2.1.5 Counterfactual Regret Minimization

Counterfactual Regret Minimization(CFR) is an iterative algorithm that approximates the Nash equilibria commonly used on imperfect information games like poker and Stratego [23, 36]. The Nash equilibria are what CFR approximates to make the best possible decision with minimal regret. Regret in this scenario is the what-ifs of the algorithm if the algorithm chooses the other decision. For CFR to make decisions, it repeatedly minimizes regret to approximate the Nash equilibria [23]. Other works have made variations of the base CFR, showing improved convergence rate, but require a significant amount of effort by researchers. A framework is proposed named AutoCFR, instead of relying on manually crafted algorithms, it automatically designs new CFR variants using an outer loop to search over algorithm designs and an inner loop to test them on equilibrium finding tasks [35].

2.2 RNN

Recurrent Neural Network is a type of Neural Network designed to process sequential data. Unlike common Neural Networks, RNNs have a memory because the information passed to the next step is based on the memory from the previous step. RNN can be applied to imperfect information games such as Stratego because of its memory. By using the memory to remember the pieces that have been revealed, the decision could be optimized [27].

2.2.1 LSTM

Long Short-Term Memory is a popular RNN variant that has the capabilities of RNN but also processes data with long-term dependencies [2]. LSTM mitigates the vanishing gradient problem that RNN faces. The vanishing gradient problem is basically as in the name, vanishing gradient, as color in a gradient, the message in this case starts strong and gradually fades away when it passes through each layer, making the learning weak or close to zero. LSTM can be used to solve imperfect information games because of its capability to process data with long-term dependencies while having the memory of RNN.

2.2.2 Aaren

Aaren, known as Attention as a recurrent neural network, combines the parallel training efficiency of Transformers with the streaming update efficiency of RNNs [13]. Aaren, like transformers, can be trained in parallel as well as RNN, can process sequential data with memory. In the study, Aaren was tested in event forecasting, time series forecasting, and classifications. Aaren achieved similar accuracy to Transformers but was more time and memory-efficient.

2.3 Stratego

The origin of Stratego can be traced to a traditional board game called Jungle, where instead of human pieces instead it used animals. In the jungle, the pieces are not hidden, making it a perfect information game. Stratego is a two-player board game where players each have a 40-piece army. In the tabletop version of Stratego, one player is assigned to the blue army and the other player is assigned to the red army. Each army has a flag assigned to it, which cannot be moved when

the game starts. There are a total of 80 troop pieces, while the total number of variations is 12. The pieces rank range from 1 to 10, and a piece that's not numbered is a bomb and the flag. The player also has time to move pieces before the game starts to make use of the fog of war and create creativity and optimal positions on the board. As the pieces have ranks, the higher the number, the higher the power. There is an instance where the lower-ranked can defeat a higher-ranked ranked, which is the piece that has a rank of one, named the spy, can defeat the strongest piece that has a rank of ten, named the marshal, if the spy attacks first. As for the bombs, almost all pieces are vulnerable except the miner, the only one who can defuse the bomb. And the pieces that cannot be moved are the bomb and the flag. Another piece that has a special ability is the scout; it can move either horizontally or vertically, not just one square. If both pieces have the same number, and the piece attacks, both of them would be eliminated. There are multiple win conditions in Stratego. The obvious one would be capturing the flag, another one would be that the opponents have no mobile pieces, another would be that there are no miners that can defuse the bomb to capture the flag, and lastly, the opposing player had the time run out.

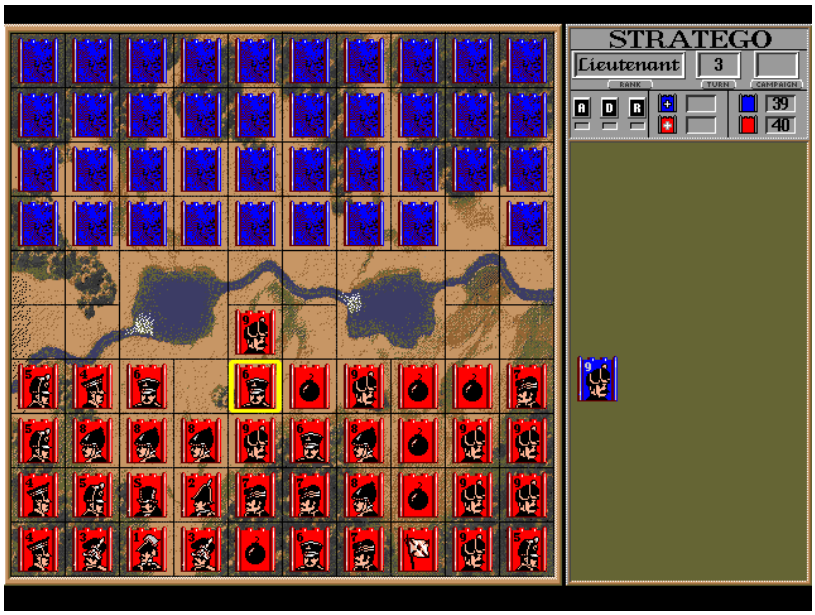

Figure 2.1: Stratego Game board.

2.4 Board Games in Community Building

Modern board games are becoming more widespread, expanding their market share each year. During COVID-19 lockdowns, board game sales increased because people were stuck at home with family and had free time. Also, playing board games during that time helped with stress, isolation, and anxiety. Certain games like Pandemic, a cooperative strategy board game where players work together to stop global disease outbreaks by discovering cures before time runs out, became more popular due to their reflection of the real-world scenario. Board game communities differ between the West and the East [14]. From the east, board games are considered social activities, while in the west, board games grew as hobbies, communities focusing on the gameplay and mechanics. In Hong Kong, there are two distinct board game communities named BGHK and Oasis. BGHK consists of Cantonese-speaking individuals who treat board games as social bonding. They even event sessions for "party game days" and "welcome newbies," emphasizing making friends rather than the game itself. While the Oasis group focuses on the hobby, playing newly released and complex games. Attracting players with the mechanics and novelty rather than just socializing. A study has been made to use board games as an answer to the problem of difficulty in creating a sense of "togetherness or kinship," and board games have advantages for improving the cognitive function of older people [3]. In the study, the Tresna Werdha Budi Pertiwi Social Home has a problem of having a sense of family among residents. By playing together, the elderly had consistent interaction with each other, reducing the feeling of loneliness and detachment. Board games not only provide entertainment but also memory stimulation and improve social bonds. Board games provide a face-to-face interaction requiring people to sit together at a table, unlike in video games, where players are often separate from each other [29]. Every shared experience creates a unique microenvironment where players enter a separate reality governed by the game's rules. In the space, the players share emotions and strategy. Sharing these emotions helps strengthen friendships, family bonds, and trust. Many board game players first connect through gaming with friends and family, which can branch out to dedicated groups or conventions, and expand their social circle.

2.5 Other Applications

An extension of MARL with DQN was demonstrated in robotics. In these environments, robots are trained to make decisions based on rewards. The study assigned different energy costs to each

robotic arm, allowing the robot to learn how to adjust its movement based on cost, using less energy when the reward is small and exerting more effort if the reward is appropriate [25]. MARL with Deep Q-learning was also used in traffic signal control. Applied a cooperative multi-agent DQN framework to manage six interconnected intersections, where each intersection acted as an independent learning agent. Using the SUMO simulator, each agent selected signal phases to minimize congestion and improve overall traffic flow. Their approach introduced a reward function based on the standard deviation of stopping vehicles across lanes, showing balanced lane utilization rather than simply reducing queue lengths. This design showed that MARL with DQN can enhance not only traffic efficiency but also environmental sustainability by reducing emissions and fuel consumption. In businesses in developing countries, it is essential that growth should be the top priority; however, this creates a problem in increasing taxation for improving infrastructure to further support the current population. Using MARL, we can create a model system where multi-agent RL acts as different parties that discover the optimal tax policies without needing to rely on traditional economic models [41].

2.6 Application to Other Game Environments

Although this study focuses on Stratego as the game environment, the MARQ framework can also be applied to other strategic and simulation-based games that involve decision-making and hidden information. Sports management titles such as Madden NFL's Dynasty Mode, NBA 2K's franchise mode, Esports Manager, and Football Manager feature complex systems of player development, transfer, and long-term planning, where agents must act without complete information about the future player performance and market behavior. In these environments, MARQ could model AI agents capable of learning optimal trade strategies or budget allocations through reinforcement learning. The framework's belief-state modeling would allow the AI to check hidden player attributes or opponent strategies, while the Deep Q-Network could optimize sequential decisions across multiple simulated seasons.

Similarly, the framework could be applied to a game called Clash Royale, where players must make fast tactical decisions without full knowledge of their opponent's hand or resource level. The Probabilistic Belief State in this setting could estimate the likelihood of specific opponent cards being available, while the Deep Q-Network would select optimal actions for troop deployment and

timing. The KLUSS component of the framework could divide the match into smaller subgames, such as lane control or counter-attack scenarios.

Chapter 3

Theoretical Frameworks

The main idea of MARQ, a Multi-Agent Rainbow Deep Q-Networks, is to employ self-play techniques and deep reinforcement learning to train agents capable of handling Stratego's large game state space and imperfect information. MARQ uses a league-based self-play system where agents learn by competing against a diverse pool of opponents, including their own previous versions and exploratory variants.

This approach addresses the fundamental challenge of Stratego: reasoning about hidden information without computing intractable common-knowledge sets. While the number of possible game states exceeds 10^{535} and initial setups alone reach 10^{66} per player [28], MARQ manages this complexity through belief state management, probabilistic reasoning, and strategic pattern recognition. The framework combines deep reinforcement learning for value estimation, attention mechanisms for processing game history, and controlled strategy mixing to prevent exploitation, while maintaining computational tractability even in this vast imperfect-information domain.

3.1 MARQ Architecture Integration

The MARQ framework operates through the hierarchical integration of its components. The system starts the observation processing, where the AAREN module ingests raw game positions to construct and maintain a probabilistic belief state(PBS). This belief state forms the base of information used for all subsequent reasoning.

Finally, the entire architecture is embedded within a league-based training paradigm. This

involves the simultaneous training and evaluation of MARQ agents.

3.1.1 Convergence and Theoretical Guarantees

The league training system ensures improvement through:

- Maintaining best response values against historical opponents
- Requiring a positive win rate against previous versions before replacement
- Diversity bonuses that prevent convergence to exploitable strategies

These properties, combined with extensive validation, provide confidence that MARQ will achieve strong, robust play despite the theoretical compromises necessary for tractability.

3.2 MARQ Policy Definition

Policy optimization is the core process of iteratively improving an agent's decision-making strategy to maximize its reward. In multi-agent environments with imperfect information, this requires policies that are not only effective but also robust to the uncertainty and adaptive behaviors of opponents, which if not done correctly can create sporadic increase or decrease in the policy loss. The main reward and loss will be determined in multiple dimensions, and due to having multiple systems from AAREN, it will create a PBS for the DQN to have improve using the comparison of the real position to the PBS positions. The DQN will be mainly rewarded by piece taken, piece saved, and winrate.

3.2.1 Core Policy Framework

A policy in MARQ, denoted as $\pi_i(a|I_i)$, defines an agent's strategy for choosing actions based on its current imperfect information set I_i rather than the complete state s_i. The main fundamental challenge of imperfect information games is that agents must make decisions without knowing the opponent's hidden information, such as piece positions and identities.

The policy operates on belief states B_t, which represent probability distributions over possible opponent configurations:

$$\pi_i(a|I_i, B_t) = P(\text{action} = a|\text{information set } I_i, \text{belief state } B_t) \tag{3.1}$$

Table 3.1: Key Symbols and Descriptions from the MARQ Framework

Symbol	**Description**	**Example (MARQ Framework)**
$\pi_i(a\|I_i)$	Policy for agent i choosing action a given information set I_i	Probability of moving a piece forward given current visible board state and belief about opponent pieces
$\mathcal{B}_t$	Belief state at time t representing probability distributions over opponent configurations	Probability distribution over possible identities and positions of hidden opponent pieces
$v_i^{\pi}(h, \mathcal{B}_t)$	Value function for agent i following policy π given history h and belief state $\mathcal{B}_t$	Expected reward for current board position considering uncertainty about opponent setup
$Q_{\text{network}}(s', a)$	Deep Q-Network approximation of action-value function	Neural network estimate of value for attacking with a suspected Marshal vs opponent piece
$R^{\text{capture}}i, t+1$	Immediate reward from piece captures weighted by piece values	+9 points for capturing opponent Marshal, -1 for losing a Scout
$O = \{o_1, \ldots, o_t\}$	Observation sequence processed by AAREN attention mechanism	Sequence of board states, moves, and attack outcomes throughout the game
α_i	Attention weights for historical observations in AAREN	Higher weight on turn when opponent's Marshal was revealed, lower on routine Scout moves
π^*	Nash equilibrium policy profile	Optimal mixed strategy that cannot be exploited by opponent strategy changes
$Q_{eval}(\mathcal{B})$	PBS evaluator network output (quality score)	Confidence score for belief distribution accuracy
$H(\pi)$	Policy entropy measure	Ensures minimum unpredictability to prevent exploitation
ϵ	Exploration rate for agent behavior	Controls exploration vs exploitation balance in multi-agent learning
γ	Discount factor for future rewards	Importance of long-term vs immediate rewards in value estimation

PBS-to-Reality Optimization

To optimize belief predictions, the Public Belief State is compared against revealed ground truth positions. When pieces are revealed, the prediction error is measured:

$$\mathcal{E}_{\text{pred}}(t) = \sum_{p \in \text{Pieces}_{\text{revealed}}(t)} D_{\text{KL}}\left(\text{PBS}_t(p) \| \delta_{s_p^*}\right) \tag{3.2}$$

where $\delta_{s_p^*}$ is the Dirac delta distribution centered on the true position s_p^* of piece p at the time of revelation.

This prediction error drives policy optimization through a belief accuracy reward:

$$R_{\text{accuracy}}(t) = -\mathcal{E}_{\text{pred}}(t) = -\sum_{p \in \text{Pieces}_{\text{revealed}}(t)} \left[\text{PBS}_t(p = s_p^*) > 0\right] \cdot \log \text{PBS}_t(p = s_p^*) \tag{3.3}$$

The policy is then optimized to maximize both game outcomes and belief accuracy:

$$\pi_i^* = \arg\max_{\pi_i} \mathbb{E}_{\tau \sim \pi_i}\left[\sum_{t=0}^{T} \gamma^t \left(R_{\text{game}}(t) + \eta \cdot R_{\text{accuracy}}(t)\right)\right] \tag{3.4}$$

where $\eta \in \mathbb{R}^+$ balances immediate game rewards with long-term belief quality.

Belief Calibration Loss To further refine PBS predictions, a calibration loss measures systematic biases when comparing predicted belief distributions to actual revealed positions across multiple episodes:

$$\mathcal{L}_{\text{calib}} = \frac{1}{|\mathcal{D}|} \sum_{(t,p) \in \mathcal{D}} \left|\text{PBS}_t(p) - \mathbb{1}[s_p = s_p^*]\right|_2^2 \tag{3.5}$$

where $\mathcal{D}$ is the dataset of all revealed pieces across training episodes, and $\mathbb{1}[s_p = s_p^*]$ is the indicator vector with 1 at the true position and 0 elsewhere.

3.2.2 Bellman Equation

The recursive value decomposition adapts to belief state uncertainty:

$$v_i^\pi(I, B_t) = \mathbb{E}_\pi[R_{i,t+1}^{\text{capture}} + \lambda_1 R_{i,t+1}^{\text{info}} + \lambda_2 R_{i,t+1}^{\text{position}} + \gamma v_i^\pi(I_{t+1}, B_{t+1}) | I_t = I, B_t] \tag{3.6}$$

where R^{capture} is the reward from the pieces captured weighted by piece values, R^{info} is the reward for the information gained for revealing opponent pieces, R^{position} measures territorial control and piece mobility assessment, and B_{t+1} is the updated belief state after observing action outcomes.

3.2.3 Nash Equilibrium in Imperfect Information

The goal remains finding Nash Equilibrium policies π^*, adapted for imperfect information:

$$v_i(\pi^*) = \max_{\pi_i} v_i(\pi_i, \pi^*_{-i}) \tag{3.7}$$

3.2.4 Implementation Framework

The attention-based memory system processes observation sequences $O = \{o_1, \ldots, o_t\}$, computes attention weights $\alpha_i = \text{softmax}(W_q h_t \cdot W_k o_i)$, and updates belief states based on historical patterns. Multi-agent learning employs main agents using current best mixed strategies, exploration agents with ϵ-greedy variants discovering new approaches, and historical pools of previous versions preventing strategy forgetting.

3.2.5 Convergence Properties

For any $\epsilon > 0$, the average strategy profile $(\bar{x}, \bar{y})$ converges to an ϵ-Nash equilibrium of the constructed subgame as $T \to \infty$. League training ensures monotonic improvement through best response maintenance, positive win rates against previous versions before replacement, and diversity bonuses preventing exploitable strategy convergence.

This framework demonstrates that imperfect-information reasoning can achieve strong and robust play while maintaining computational tractability on consumer hardware.

3.3 Multi-Piece Coordination and Pattern Recognition

The MARQ framework incorporates advanced pattern recognition capabilities to identify coordination between opponent pieces, providing additional strategic insights beyond individual piece behavior.

3.3.1 Coordination Pattern Detection

The system identifies strategic coordination through spatial proximity. Pieces are considered coordinated if they maintain a Manhattan distance of ≤ 2 during movement phases:

$$d_{spatial}(p_i, p_j, t) = |x_i(t) - x_j(t)| + |y_i(t) - y_j(t)| \tag{3.8}$$

$$\text{Coordination}(p_i, p_j) = \mathbb{I}\{d_{spatial}(p_i, p_j, t) \leq 2\} \tag{3.9}$$

This binary coordination signature feeds into the PBS update mechanism to adjust beliefs for clustered pieces.

3.3.2 Pattern-Based Belief Updates

Coordination patterns influence belief state updates for all pieces in a coordinated group:

$$\mathcal{B}_{coordinated}(p_k) = \mathcal{B}_{individual}(p_k) + \sum_{p_j \in \text{Group}} \omega_{kj} \cdot \Delta\mathcal{B}(p_j) \tag{3.10}$$

where ω_{kj} represents the influence weight between coordinated pieces.

3.3.3 Behavioral Pattern Classification

The framework classifies observed behaviors into strategic patterns:

- **Aggressive Coordination**: High-value pieces supporting attacks
- **Defensive Formations**: Pieces protecting key assets (flag, high-value pieces)
- **Scout Networks**: Coordinated reconnaissance and information gathering
- **Bait Strategies**: Coordinated sacrifice patterns for information gain

Each pattern triggers specific belief updates and strategic adjustments in the PBS.

3.3.4 Temporal Pattern Recognition

The system maintains temporal coordination histories to identify evolving strategic patterns:

$$\text{TemporalCoord}(t) = \sum_{i=1}^{t} \gamma^{t-i} \cdot \text{CoordinationEvidence}(i) \tag{3.11}$$

This allows the framework to distinguish between coincidental coordination and deliberate strategic patterns.

3.4 Game Environment and State Management

The Stratego environment provides the foundation for agent training and evaluation, managing the complex dynamics of imperfect information and simultaneous move selection.

3.4.1 Information Set Management

The environment maintains distinct representations for different information contexts:

$$\mathcal{I}_{player} = \{s : s \text{ is consistent with player observations}\} \tag{3.12}$$

Each player receives only the subset of game state information available through their observation function.

3.4.2 State Space Complexity

The theoretical state space of Stratego demonstrates the challenge addressed by MARQ:

$$|\mathcal{S}| = \binom{40}{1, 1, 2, 3, 4, 4, 4, 5, 8} \times 10^{10} \times 2^{40} \approx 10^{535} \tag{3.13}$$

This high complexity necessitates knowledge-limited reasoning and belief state management.

3.4.3 Temporal State Evolution

Game states evolve according to the update function:

$$s_{t+1} = \text{EnvStep}(s_t, a_t^1, a_t^2, \theta_t) \tag{3.14}$$

where θ_t represents stochastic elements (hidden information revelation, time limits, etc.).

3.4.4 Valid Action Filtering

The environment enforces game rules through valid action constraints:

$$\mathcal{A}_{valid}(s, player) = \{a : \text{IsLegalMove}(a, s, player) \wedge \text{RespectsGameRules}(a, s)\} \tag{3.15}$$

This ensures that agents can only select legal moves consistent with Stratego's rules and piece capabilities.

3.4.5 Piece Setup and Initialization

The initial game setup involves strategic piece placement governed by setup policies:

$$\pi_{setup}(\text{placement}|player) = \text{NeuralNetworkSetup}(\text{strategic\_features}, \theta_{setup}) \tag{3.16}$$

where strategic features include positional value maps, piece interaction potentials, and defensive vulnerabilities.

The setup phase represents a critical strategic decision that influences the entire game trajectory, making it a key component of the overall MARQ training framework [10].

3.5 Theoretical Convergence Guarantees

The MARQ framework provides theoretical guarantees for convergence and performance under specific conditions.

3.5.1 Incremental Learning Bounds

Under standard RL assumptions (independent identical distribution of experiences, bounded rewards), the MARQ algorithm provides the following bound on value function estimation:

$$\|V_t - V^*\|_\infty \leq \frac{C}{\sqrt{t}} + \epsilon_{approx} \tag{3.17}$$

where C depends on reward bounds and discount factor, and ϵ_{approx} represents function approximation error.

3.5.2 Belief State Accuracy Bounds

The PBS accuracy improves with additional observations according to:

$$\mathbb{E}[\text{Accuracy}(\mathcal{B}_t)] \geq 1 - \exp(-\alpha \cdot N_{obs}(t)) \tag{3.18}$$

where $N_{obs}(t)$ is the cumulative number of informative observations and α is a positive constant.

3.5.3 Strategy Convergence in Self-Play

The league training methodology ensures convergence to ϵ-Nash equilibria under standard monotonic improvement assumptions:

$$\lim_{T \to \infty} \|\pi_T - \pi^*\| \leq \epsilon \tag{3.19}$$

for some $\epsilon > 0$ depending on the exploration parameters and opponent diversity.

These theoretical foundations provide confidence in the MARQ framework's ability to learn robust, effective strategies in the challenging domain of imperfect information strategic gameplay.

3.6 Deep Q-Network with Belief States

Deep Q-Networks in MARQ operate on belief states rather than complete game states. The network approximates Q-values for probability distributions over opponent configurations, $\mathcal{B}_t$:

$$Q(s, a) = \mathbb{E}_{s' \sim \mathcal{S}_t \subseteq \mathcal{B}_t}[Q_{network}(s', a)]$$

The system uses a **Standard Replay Buffer** for experience storage, sampling transitions $(\mathcal{B}_t, a_t, r_t, \mathcal{B}_{t+1})$ uniformly to train the value function.

3.6.1 Rainbow DQN Components

The implementation incorporates key enhancements from the Rainbow DQN architecture to improve stability and sample efficiency:

Distributional RL (Categorical DQN)

Stratego's inherent uncertainty leads to multimodal value distributions. We model the value distribution $Z(s, a)$ directly rather than just its expectation. The distribution is approximated by a discrete distribution over $N = 51$ atoms $\{z_i\}_{i=1}^{N}$, with support ranging from V_{min} to V_{max}:

$$Z_\theta(s, a) = \sum_{i=1}^{N} p_i(s, a)\delta_{z_i} \tag{3.20}$$

The training objective minimizes the Kullback-Leibler divergence between the predicted distribution and the projected target distribution:

$$D_{KL}(\Phi\hat{\mathcal{T}}Z_{\theta'}||Z_\theta) \tag{3.21}$$

where Φ is the projection operator mapping the Bellman update onto the fixed support atoms.

Noisy Networks for Exploration

To ensure consistent exploration without the need for discrete ϵ-greedy schedules, we employ Noisy Linear layers. The weights and biases are perturbed by a deterministic function of factorized Gaussian noise:

$$y = (\mu_w + \sigma_w \odot \epsilon_w)x + (\mu_b + \sigma_b \odot \epsilon_b) \tag{3.22}$$

where ϵ_w, ϵ_b are random variables. This allows the network to learn the degree of exploration required for different states.

Dueling Network Architecture

The network explicitly separates the estimation of state values $V(s)$ and advantage values $A(s, a)$ using two separate streams that share a common convolutional feature extractor:

$$Q(s, a; \theta, \alpha, \beta) = V(s; \theta, \beta) + \left(A(s, a; \theta, \alpha) - \frac{1}{|\mathcal{A}|} \sum_{a'} A(s, a'; \theta, \alpha) \right) \tag{3.23}$$

This decomposition leads to better policy evaluation in the presence of many similar-valued actions.

3.6.2 Exploration-Driven Q-Value Adjustment

We augment the Q-values with an exploration bonus based on the uncertainty of the belief state, utilizing an "optimism in the face of uncertainty" approach:

$$Q_{augmented}(s, a) = Q_{base}(s, a) + \lambda \cdot U(a) \tag{3.24}$$

where $U(a)$ represents the entropy of the belief distribution at the target location, encouraging the agent to investigate unknown opponent pieces.

3.6.3 Adaptive Learning Rate Scheduling

MARQ employs adaptive learning rate adjustment based on loss trends and performance metrics. The learning rate η_t is dynamically adjusted according to:

$$\eta_{t+1} = \begin{cases} \eta_t \cdot \alpha_{decay} & \text{if } \mathcal{L}_{recent} > \mathcal{L}_{baseline} \cdot \tau_{increase} \\ \eta_t \cdot \alpha_{increase} & \text{if } \mathcal{L}_{recent} < \eta_{threshold} \\ \eta_t \cdot \alpha_{time} & \text{time-based decay} \end{cases} \tag{3.25}$$

where $\mathcal{L}_{recent}$ is the average loss over recent training steps, $\tau_{increase}$ is the increase threshold, and α parameters control the rate of adjustment.

3.6.4 Cross-System Performance Monitoring

The framework includes comprehensive performance monitoring to detect misalignment between PBS and DQN components. Key metrics include:

- PBS prediction accuracy trends
- DQN loss convergence patterns
- Action prediction alignment scores
- Reward distribution analysis

Misalignment is detected when PBS accuracy falls below $\theta_{pbs} = 0.5$ while DQN loss exceeds $\theta_{dqn} = 0.5$, triggering corrective measures in the training process.

3.7 Attention as Recurrent Neural Network (AAREN)

The AAREN module acts as the core memory and belief state system within the MARQ architecture. It is specifically designed to process extended sequences of partial observations in Stratego, integrating information over hundreds of moves while avoiding the vanishing gradient problem that affects recurrent models like LSTMs or GRUs in long tasks. AAREN leverages an attention mechanism to provide direct, weighted access to the entire history of observations, enabling more coherent and computationally efficient belief tracking [13].

3.7.1 AAREN Architecture

AAREN implements an attention-based recurrent computation with state triplet (a_t, c_t, m_t) where:

- a_t: Weighted sum of encoded values
- c_t: Normalization constant
- m_t: Cumulative maximum for numerical stability

Formally, given a sequence of observations $O = o_1, ..., o_t$ where each o_i encapsulates the visible board state and public action outcomes, the AAREN cell computes:

$$k_t = W_k(o_t), \quad v_t = W_v(o_t) \tag{3.26}$$

$$s_t = q^T k_t \tag{3.27}$$

$$m_t = \max(m_{t-1}, s_t) \tag{3.28}$$

$$a_t = a_{t-1} \exp(m_{t-1} - m_t) + v_t \exp(s_t - m_t) \tag{3.29}$$

$$c_t = c_{t-1} \exp(m_{t-1} - m_t) + \exp(s_t - m_t) \tag{3.30}$$

$$h_t = \frac{a_t}{c_t} \tag{3.31}$$

where W_k and W_v are learned projection matrices, q is a learned query vector, and h_t is the output representation.

3.7.2 Parallel Prefix Scan Implementation

For training efficiency, AAREN employs parallel prefix scan computation across the observation sequence. This allows processing of entire sequences in parallel while maintaining the O(1) per-timestep inference complexity of traditional RNNs.

The parallel implementation computes:

$$\mathbf{s} = \text{cumsum}(\exp(\mathbf{q}^T\mathbf{k} - \mathbf{m})) \tag{3.32}$$

$$\mathbf{a} = \text{cumsum}(\mathbf{v} \odot \exp(\mathbf{q}^T\mathbf{k} - \mathbf{m})) \tag{3.33}$$

where cumsum denotes cumulative sum, $\odot$ is element-wise multiplication, and boldface indicates vectorized operations.

3.7.3 AAREN-Based Piece Value Inference

The AAREN model learns to infer piece values from action sequences using 24-dimensional feature vectors encoding movement patterns, contextual information, and game state features. The network outputs probability distributions over piece types:

$$P(piece\_type = t|action\_sequence) = \text{softmax}(W_{out} \cdot h_T + b_{out})_t \tag{3.34}$$

This allows the PBS to combine AAREN predictions with rule-based inference and historical pattern matching for robust piece identification under uncertainty.

3.8 PBS Quality Evaluation Framework

The MARQ framework incorporates a PBS evaluation system that assesses prediction quality and provides feedback for continuous improvement. This system addresses the challenge of determining when and how much to trust PBS predictions.

3.8.1 PBS Evaluator Network

The PBS Evaluator employs a neural network architecture that takes belief distributions as input and outputs quality scores representing prediction confidence:

$$Q_{eval}(\mathcal{B}) = \text{MLP}(\mathcal{B}; \theta_{eval}) \tag{3.35}$$

where $\mathcal{B}$ is the belief distribution over piece types and θ_{eval} represents network parameters. The evaluator learns to predict the expected reward of PBS predictions based on historical outcomes.

3.8.2 Reward-Based Quality Assessment

The evaluator's training objective is based on rewards computed from ground truth comparisons:

$$R_{quality}(\mathcal{B}, g) = \alpha \cdot \mathcal{B}[g] - \beta \cdot |V_{pred} - V_{actual}| - \gamma \cdot \text{distance\_penalty} \tag{3.36}$$

where:

- g is the ground truth piece type
- $V_{pred} = \sum_t r_t \cdot \mathcal{B}[t]$ is the expected piece value
- V_{actual} is the actual piece rank
- α, β, γ are weighting parameters

Higher-value pieces receive greater reward/penalty weighting, reflecting their strategic importance.

3.8.3 Bias Detection and Correction

The system includes bias tracking that identifies systematic over/under-prediction patterns for different piece types. For each piece type t, a correction factor ϕ_t is computed:

$$\phi_t = \frac{\text{actual\_rate}_t}{\text{predicted\_rate}_t} \cdot \text{accuracy\_adjustment}_t \tag{3.37}$$

Corrected belief distributions are then computed as:

$$\mathcal{B}_{corrected}[t] = \phi_t \cdot \mathcal{B}[t] \tag{3.38}$$

3.8.4 Feature Importance Learning

The framework learns which action features are most predictive for PBS inference through a dedicated neural network:

$$\mathbf{w}_{importance} = \text{FeatureNet}(\mathbf{x}_{action}; \theta_{feat}) \tag{3.39}$$

where $\mathbf{x}_{action}$ is the 24-dimensional action feature vector. The importance weights $\mathbf{w}_{importance}$ are used to enhance feature selection during inference.

3.8.5 Active Learning for Information Gathering

The system determines when additional information gathering is beneficial through uncertainty assessment:

$$\text{NeedMoreInfo}(\mathcal{B}) = \mathbb{1}\{H(\mathcal{B}) > \theta_H\} \cdot \mathbb{1}\{Q_{eval}(\mathcal{B}) < \theta_Q\} \tag{3.40}$$

where $H(\mathcal{B})$ is the entropy of the belief distribution, θ_H is the uncertainty threshold, and θ_Q is the quality threshold.

3.9 Specialized Agent Architectures

3.9.1 Setup Agent via Distributional RL

The game's initial setup is handled by a specialized 'SetupAgent' that learns optimal piece configurations. It treats the placement phase as a sequential decision process.

- **State Representation**: A 3-channel input tensor encoding the Board State, Valid Positions Mask, and Current Piece Type Mask.
- **Evaluation**: The agent outputs a distributional Q-value for each of the 100 board positions, $Q(s, pos) \rightarrow$ Distribution over $[V_{min}, V_{max}]$.
- **Exploitability Critic**: To prevent creating static, easily countered setups, the agent includes a critic network that penalizes predictable placements:

$$\mathcal{L}_{total} = \mathcal{L}_{C51} + \lambda \cdot \mathcal{L}_{predictability} \tag{3.41}$$

3.9.2 Information Set Monte Carlo Tree Search (IS-MCTS)

For high-stakes decision making, particularly in the endgame, MARQ employs an Information Set MCTS agent that combines search with learned value functions.

1. **Determinization**: The agent samples a consistent concrete world state s_{det} from the PBS beliefs: $s_{det} \sim P(S|\mathcal{B}_t)$.
2. **Tree Traversal**: The search tree is traversed using a UCB1 variant to balance exploration and exploitation of move sequences.
3. **Leaf Evaluation**: Instead of random rollouts, leaf nodes are evaluated using the DRQN's value function, providing a high-quality heuristic estimate:

$$V(s_{leaf}) \approx Q_{DRQN}(s_{leaf}, a_{best}) \tag{3.42}$$

This hybrid approach allows deep lookahead while respecting the uncertainty inherent in the belief state.

3.10 Multi-Agent Reinforcement Learning

MARQ employs a league reinforcement learning system that minimizes non-stationarity, a fundamental challenge in multi-agent learning where policy updates by one agent alter the environment experienced by others. The system mitigates this through several mechanisms:

3.10.1 League-Based Training Architecture

The league consists of a dynamic pool of agents to ensure robust generalizability:

- **Main Agent**: The current best-performing policy being actively optimized.
- **Historical Pool**: A saved collection of past Main Agent checkpoints (snapshots) that represent distinct strategic eras.

During training, opponents are sampled using a 50/50 distribution:

$$P(\text{opponent}) = \begin{cases} 0.5 & \text{Main Agent (Self-Play)} \\ 0.5 & \text{Uniform(Historical Pool)} \end{cases} \tag{3.43}$$

This prevents "cycling" where the agent re-learns and un-learns the same strategies endlessly, forcing it to defeat all previous versions of itself.

3.10.2 Experience Replay

The framework employs a **Standard Replay Buffer** that stores transitions $(\mathcal{B}_t, a_t, r_t, \mathcal{B}_{t+1}, \pi_t)$. Uniform sampling is used to stabilize the training distribution across the agent's history.

3.10.3 Multi-Dimensional Reward Structure

The reward function $R(s, a, s')$ is carefully shaped to guide learning in the sparse-reward Stratego environment:

$$R_{total} = R_{move} + R_{capture} + R_{tactical} + R_{info} \tag{3.44}$$

Where:

- R_{move}: Small rewards for advancing pieces (+0.05) to encourage active play.
- $R_{capture}$: Scaled by the rank of the captured piece ($0.15 \times \frac{\text{Rank}}{11.0}$).
- $R_{tactical}$: Specific bonuses for Miner vs Bomb (+0.5) and Spy vs Marshal (+1.0).
- R_{info}: Rewards for revealing high-value opponent pieces (+0.3 for Rank ≥ 9).

Game phase multipliers adjust these weights, prioritizing material in the early game and safe positioning in the endgame.

3.11 Strategy Mixing

Extensive-form imperfect information games with deterministic strategies are inherently suboptimal and vulnerable to exploitation. An adversary with knowledge of a deterministic policy effectively reduces the game to a perfect information setting, nullifying the strategic uncertainty fundamental to games like Stratego.

3.11.1 Mixed Strategy Foundation

The MARQ framework operates on mixed strategies π_{mixed} derived from deterministic policies π_{det} through controlled randomization:

$$\pi_{mixed}(a|I) = \sum_i \alpha_i \cdot \pi_{det_i}(a|I) \tag{3.45}$$

where α_i are mixing weights satisfying $\sum_i \alpha_i = 1$ and $\alpha_i \geq 0$.

3.11.2 Exploitation Resistance

To maintain resistance against opponent exploitation, the system continuously monitors policy entropy:

$$H(\pi) = -\sum_{a} \pi(a|I) \log \pi(a|I) \tag{3.46}$$

Policy updates are constrained to maintain minimum entropy levels while ensuring sufficient exploration-exploitation balance.

3.11.3 Dynamic Strategy Adaptation

Strategies adapt dynamically based on opponent modeling:

$$\pi_{adapted}(a|I, O_{opp}) = (1 - \beta) \cdot \pi_{base}(a|I) + \beta \cdot \pi_{counter}(a|I, O_{opp}) \tag{3.47}$$

where O_{opp} represents observed opponent behavior patterns and β is the adaptation coefficient.

This approach ensures robust performance against diverse opponent strategies while maintaining computational efficiency and preventing the erratic behavior that can arise from overly randomized policies [20].

3.11.4 Stratego

Stratego is an imperfect information board game with European roots, popularized in the Netherlands in the 1940s and released in North America in 1961. The game is played on a 10x10 board that includes two 2x2 "lake" areas in the center, which are impassable. Each player commands an army of 40 pieces of varying ranks. A piece may move one square to an adjacent, unoccupied space. During the game, the identity and rank of the pieces are kept secret from the opposing player. The objective is to capture the opponent's flag or eliminate all of their movable pieces.

Each army consists of 12 different types of pieces, with ranks ranging from 1, the spy, to 10, the Marshal, alongside bombs and a Flag. The standard distribution per player as follows: one Marshal (10), one General (9), two Colonels(8), three Majors (7), four Captains (6), four lieutenants (5), four Sergeants (4), five Miners (3), eight scouts (2), and one Spy (1), plus six bombs and one Flag. Higher-ranked pieces defeat lower-ranked ones during combat, except in special situations. For instance, the Spy, although the lowest-ranked, can defeat a Marshal if the Spy attacks first, while the Miner is the only piece that can defuse bombs. The scout has a unique ability that allows it to move any number of squares horizontally or vertically in a straight line. Bombs, when attacked by

any piece other than the Miner, would result in the piece attacking being defeated. And both the bomb and the Flag cannot be moved once the game starts.

A turn consists of moving one piece to an adjacent, unoccupied square (except Scouts, which can move multiple spaces). When a piece moves onto a square occupied by an opposing piece, a battle occurs: both pieces reveal their ranks, and the lower-ranked one is removed from the board. If both ranks are equal, both are eliminated. Because the identity and rank of all pieces remain hidden until battle, players must balance offense and defense while inferring their opponent's hidden setup. Victory is achieved by capturing the opponent's Flag, immobilizing all movable enemy pieces, or forcing the opponent to run out of time

Figure 3.1: Stratego Pieces

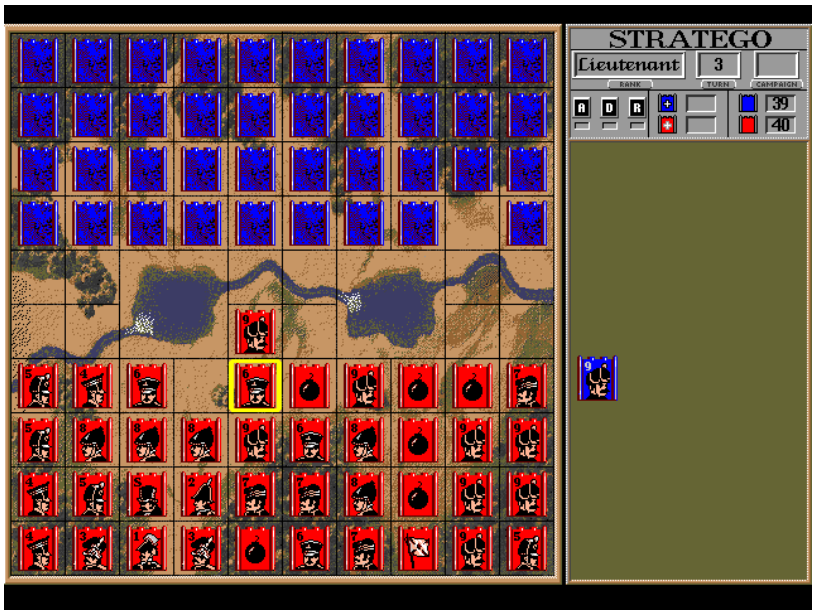

Figure 3.2: Stratego Game board.

Chapter 4

Methodology

This chapter outlines the methodology employed to evaluate the effectiveness of an AI system based on MARQ in solving mind sports with imperfect information. The approach involves the development of a competitive system, followed by the evaluation of its performance through win-rate analysis against human players and an assessment of its adaptability across diverse game environments. The subsequent sections detail the processes and methods that will be used to achieve these research objectives.

4.1 Project Planning

This research will be conducted according to a structured plan encompassing participant selection, sample size determination, and experimental procedures. The project will follow a rapid prototyping approach for the design and implementation of the AI system. The timeline is segmented into four distinct phases. The development phase will run from September 2025 to February 2026, beginning with game environment development from September through November 2025, followed by AI training from December 2025 to January 2026, and concluding with system deployment in February 2026. The user testing phase will take place in March 2026, involving one-on-one matches and data collection activities. The final analysis and documentation phase will commence in April 2026, focusing on data analysis and manuscript preparation.

4.1.1 Community Introduction to Stratego

The introduction of the game Stratego to the participant community is a prerequisite for this study. This necessity arises from a complete lack of available local participants possessing prior knowledge of the game's rules or strategies. Unlike more popular games like Chess, Stratego's niche status means a pre-existing pool of experienced players is not available for participation to test against MARQ. Therefore, a structured introduction is essential to cultivate the necessary participant cohort from the ground up. This approach ensures all participants begin with a uniform baseline understanding. It allows for the parallel development of both human expertise and artificial intelligence, creating a synchronized learning environment where the adaptive capabilities of the MARQ system can be fairly assessed against a human cohort that is also in the process of learning.

4.1.2 Participant Selection

The participant selection process is designed to recruit a cohort of 20 individuals via convenience sampling, taking into account the project's resource constraints while still yielding a enough sample. The target demographic is enthusiasts of strategic mind sports of a diverse age and game experience.

Prospective participants will be screened to evaluate two key attributes. Their strategic game experience is key to ensuring a baseline aptitude, and second, their confirmed lack of prior knowledge of Stratego to ensure the study's "synchronized learning" premise. Ultimately, only those who successfully finish the screening phase will be admitted, guaranteeing they possess the necessary foundational skills to provide meaningful interaction with the MARQ system and substantive feedback.

4.1.3 Experimental Procedures

One-on-one matches between human participants and the MARQ system will be conducted on-site using a single device. To ensure consistency, each participant will receive a checklist outlining the procedures for interacting with the game.

Each session will last one (1) hour, during which participants must complete at least two (2) games against MARQ. Additional games may be played if the time permits. The following game statistics will be recorded will include the winner, total game duration, total moves per player, and the number and type of remaining pieces for both players. Immediately following the session, each

participant will be given an evaluation form to provide open-ended feedback on their experience.

4.2 System Development

The system development consists of two components which is the game environment and MARQ. The game environment will be developed to serve as the interface for human players and the world in which the AI agent operates. While MARQ is the trained model to be used as the bot playing against the player.

4.2.1 Piece Value Computation

To inform MARQ's reward function and internal heuristics, a quantitative value was assigned to each piece in Stratego. This was achieved through a systematic process to account for the relative power of each piece.

1. Initial Unweighted Calculation: The value of each piece was first computed using the formula:

$$\text{Unweighted Piece Value} = a - b$$

   where a is the number of pieces it defeats and b is the number of pieces it loses to.

2. Normalization: The computed values were then normalized to a range of [0,1] using min-max scaling to create a standardized set of weights, typically used in chess evaluation.

3. Final Weighted Calculation: The final value was then recomputed by weighting each interaction:

$$\text{Final Piece Value} = \sum_{i \in \text{beats}} w_i \; - \sum_{j \in \text{loses to}} w_j$$

where w_i and where w_j are the normalized values of the pieces it beats or loses to, respectively. This provides a more accurate reflection of each piece's strategic importance.

4.2.2 Handling Imperfect Information: Probabilistic Belief State (PBS)

A key challenge in Stratego is the unknown identity of opponent pieces. To address this, the MARQ system will incorporate a Probabilistic Belief State (PBS). The PBS is a matrix that maintains a

probability distribution for the identity of each of the opponent's unrevealed pieces. This belief state is updated throughout the game using two forms of reasoning:

- Deductive Reasoning: When a piece interaction occurs, the PBS is updated with absolute certainty. The probabilities of the unknown piece being a Scout, Miner, or Sergeant would be set to 0, and the remaining probabilities are renormalized.
- Inductive Reasoning: The system infers piece identity from an opponent's behavior. For example, a piece that moves frequently and far is more likely to be a Scout. A piece that moves away from a threat exhibits defensive behavior, slightly increasing the probability that it is a high-value piece like the Flag or Marshal. These behavioral heuristics, based on aggression, defensiveness, and movement patterns, are used to adjust the probabilities in the PBS via Bayesian updates.
- Real state vs. PBS comparison: The system compares the PBS with the real state of the board and can be updated to allign more to the true piece for training. Training will have two stages separated, creating the belief state and later on normalizing the values from the real state to the PBS for a more accurate prediction. The main purpose of the PBS is to create a pseudo-perfect information game out of the imperfect information.

4.3 Model Training

The most time-intensive part of the research. Mainly due to the computation needed by the model to learn both exploration and exploitation. Since we are using a DQN model, the epsilon value in the first few games will be large, making the model have a higher likelihood of choosing random moves to try and explore. Once the epsilon value becomes smaller and smaller the model tends to follow what it thinks will be the optimal move.

The state representation provided to the DQN is crucial. It will be a multi-channel tensor including: (1) the location of the player's pieces, (2) the location of known opponent pieces, and (3) the complete Probabilistic Belief State (PBS) for all unknown pieces.

Every 100 episodes, the target model or history model will be synced from the policy network, along with the memory replay by the AAREN.

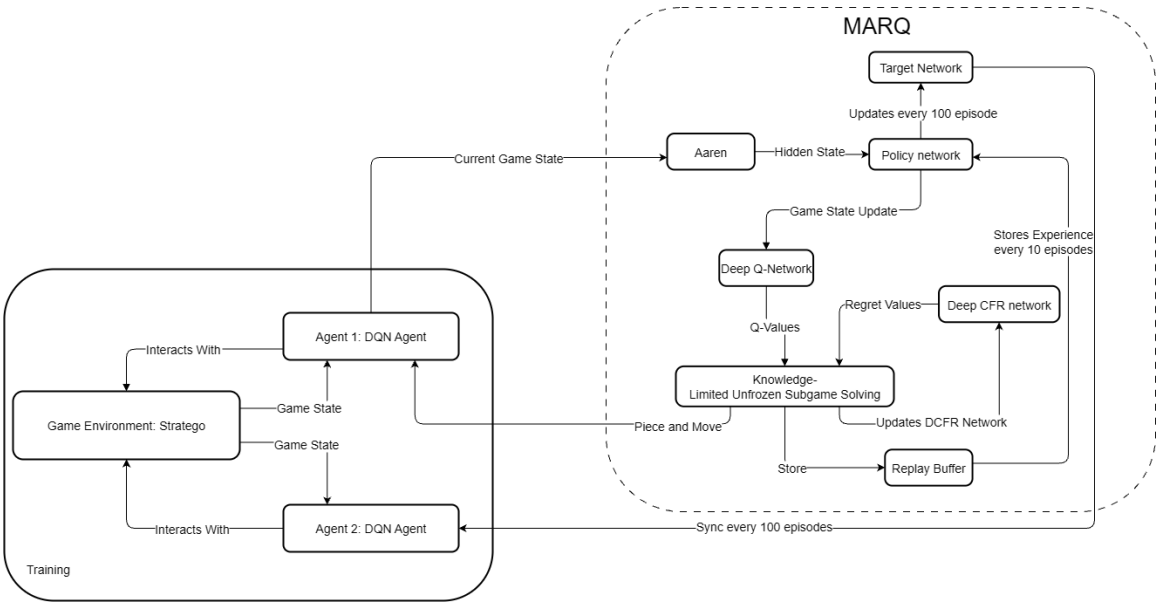

Figure 4.1: The MARQ System Architecture, which takes the Board State and PBS as input.

4.3.1 Deployment

Upon completion of the training and validation cycles, the best-performing MARQ model weights will be saved and finalized for deployment. Deploy the trained model into the Stratego game environment described in Section 2 of the methodology.

The deployed system will function as the complete application used during the user testing phase. It will be configured to run locally on the single device designated for the experimental procedures. In this production mode, the model's exploratory functions will be deactivated for a more measured approach from the agent, always selecting the action with the highest predicted Q-value. This ensures that all human participants compete against the identical, optimized version of the MARQ AI, providing a consistent baseline for the quantitative and qualitative evaluations.

In the figure above, the player starts the game and the loop begins until either the player wins by getting the flag of the agent or loses if the agent reaches the player's flag. The agent gets the current position and sends it to AAREN, constructing the possible positions of the board. After getting the PBS, it is then used to calculate the possible moves and actions that can be taken by the DeepCFR. Once the DeepCFR calculates pieces that are possible to have moves, it is then the role of the DQN to formalize a strategy to be used.

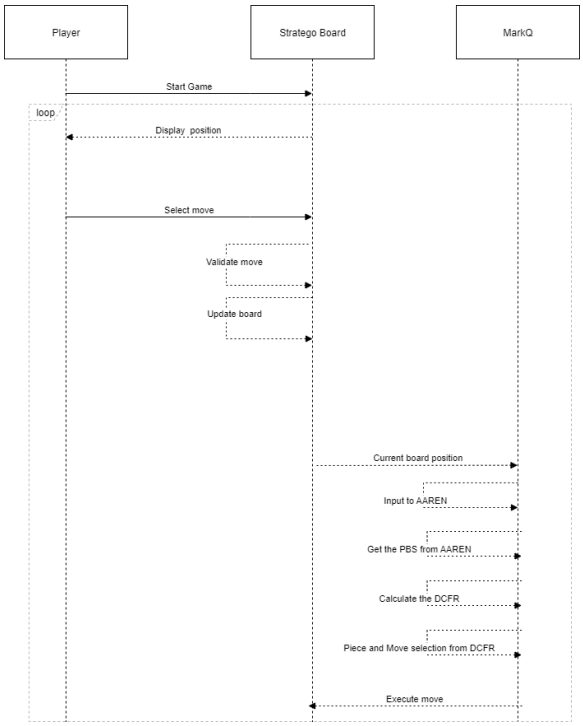

Figure 4.2: Game Sequence Diagram

4.4 Evaluation

A comprehensive evaluation approach combining quantitative and qualitative methods will be used to assess the MARQ system's performance and the user experience.

4.4.1 Quantitative Analysis

The quantitative assessment of the model's performance is conducted through the Evaluation component. Key metrics are tracked to gauge learning progress and strategic capability. Primary analytics include:

- Win Rate%: The percentage of games won against human participants.
- Average Game Length: The mean number of moves or duration of games, indicating game efficiency.
- Decision-Making Speed: The average time taken by the agent to select an action.

- Loss Curves: The progression of policy and value loss over training time, indicating the stability of the learning process.
- Player Performance over Time: To evaluate the "synchronized learning" environment, a two-tailed paired t-test will be used. This will determine if there is a statistically significant difference in gameplay metrics (e.g., difference in remaining pieces) between a player's first and last games, assessing if human players improve against the AI.

  The results from the Likert-scale questions will be analyzed using a weighted mean to derive a quantitative score for each dimension of user acceptance.

4.4.2 Qualitative Analysis

To assess the end-user opinion on usability, a usability testing session will focus on gathering participant feedback through an evaluation form. The form will evaluate three key dimensions:

- Performance: Assessing the agent's speed, consistency, and perceived strategic competence.
- Usability: Evaluating the ease of use, intuitiveness of the interface, and overall user experience.
- Relevance: Gauging the AI's value as a strategic tool and its potential for helping players learn Stratego.

4.4.3 Model Version History

The development of MARQ is an iterative process. Each model version represents a snapshot of this evolution, performance metrics, and reliability.

4.4.4 Initial Training

Under Appendix B are the initial training results of 1000 training cycles or episodes. We can see that the early training phase is not yet refined due to the agent being in the exploratory phase, ready to search and find an exploitable strategy. The reward indicates that it is having a hard time finding a good strategy to fight against the other agent. The win-rate stayed to a draw due to the agents being on an endless loop of moving pieces back in forth. Another issue is that the agent has reduced epsilon, making the moves deterministic, and only picking the "best move" with the highest q-value.

Appendix A

Gantt Chart

This section includes the images of the Gantt chart that will represent the timeline for the whole duration of this study.

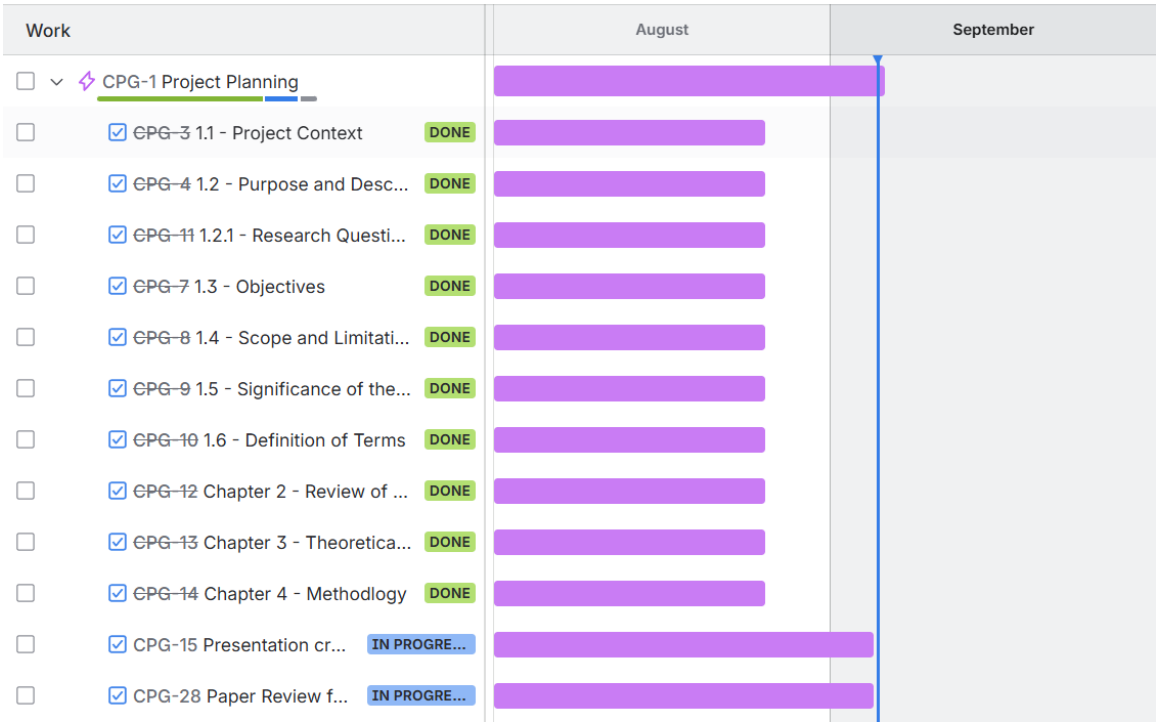

Figure A.1: Timeline: August 1 - September 5

Figure A.2: Timeline: September 11 - November 30

Figure A.3: Timeline: October 27 - January 27

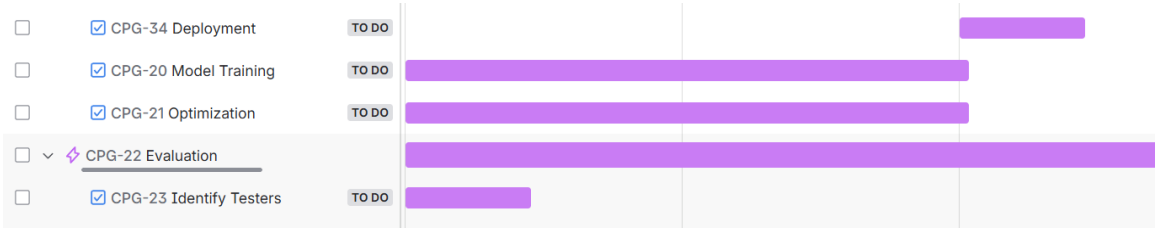

Figure A.4: Timeline: December 1 - February 14

Figure A.5: Timeline: February 15 - May 4

Appendix B

System UI and Training

This section includes the images of the system UI and Training.

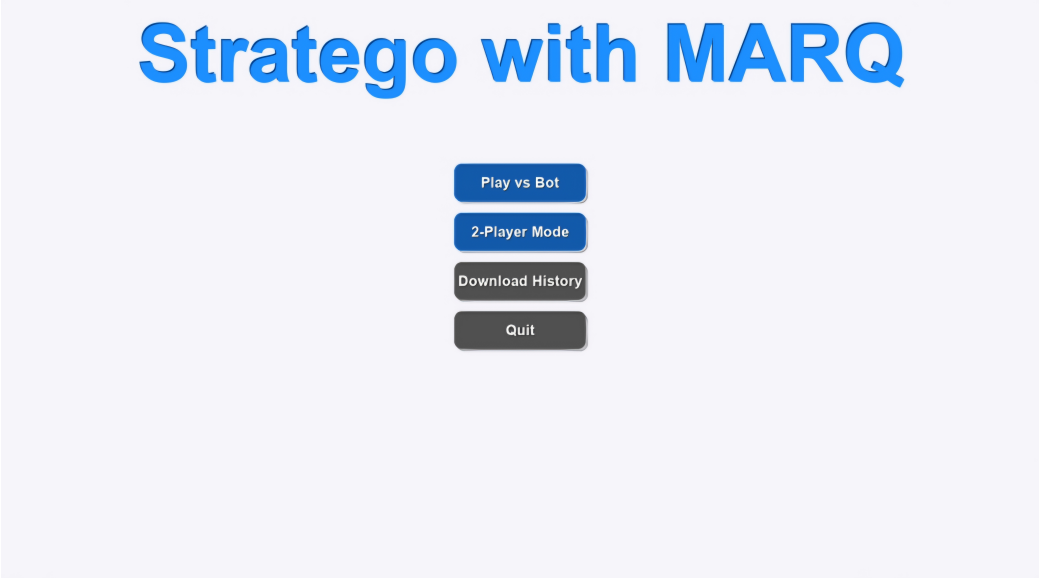

Figure B.1: Game Menu

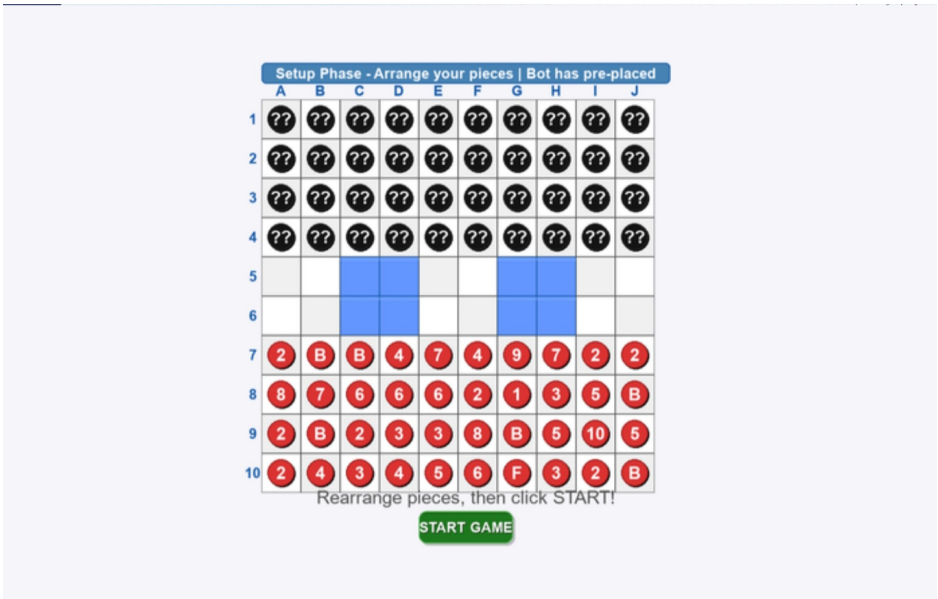

Figure B.2: Setup Phase

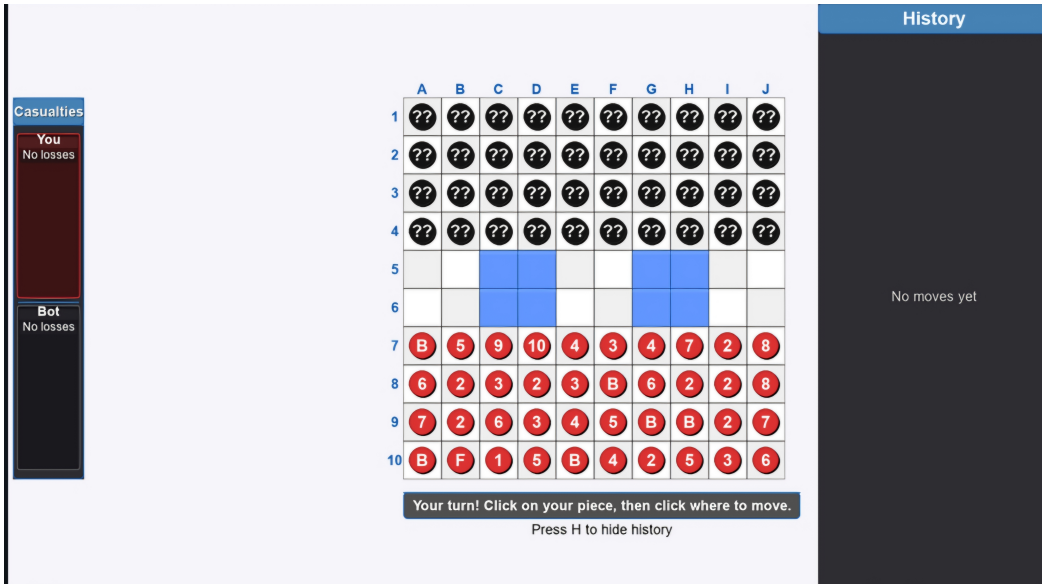

Figure B.3: Game Start

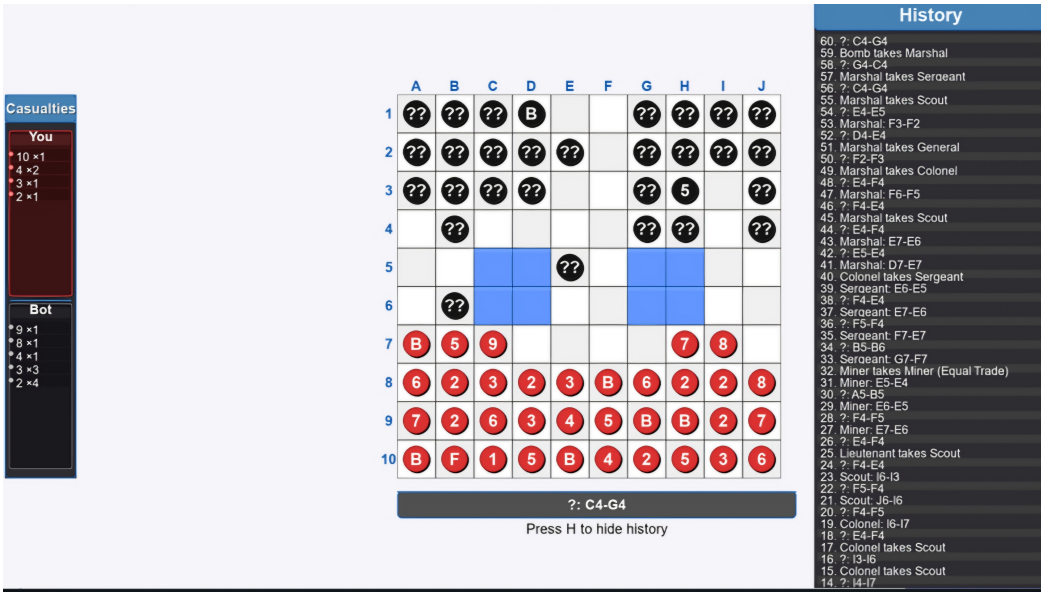

Figure B.4: Battle and History

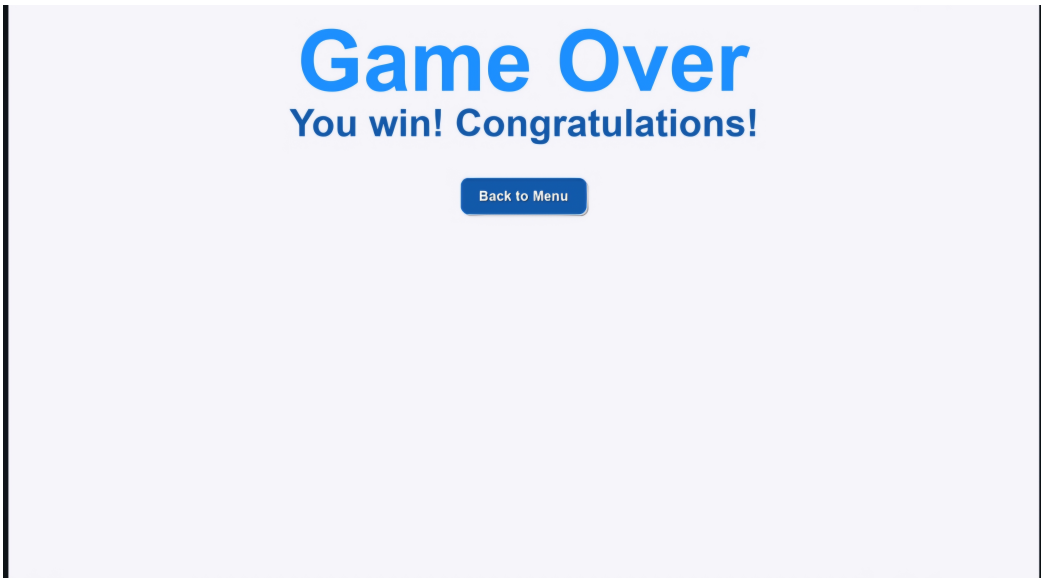

Figure B.5: Victory Screen

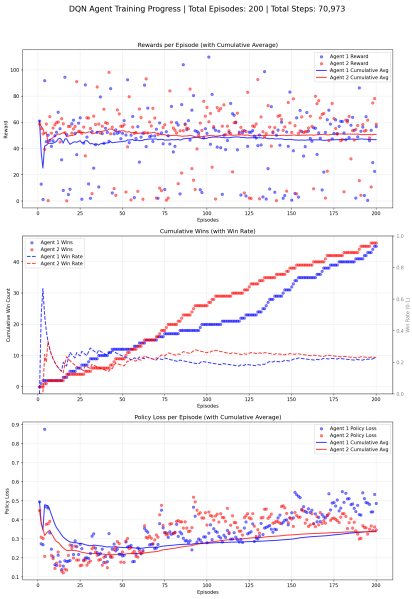

Figure B.6: Training at 200 Episodes

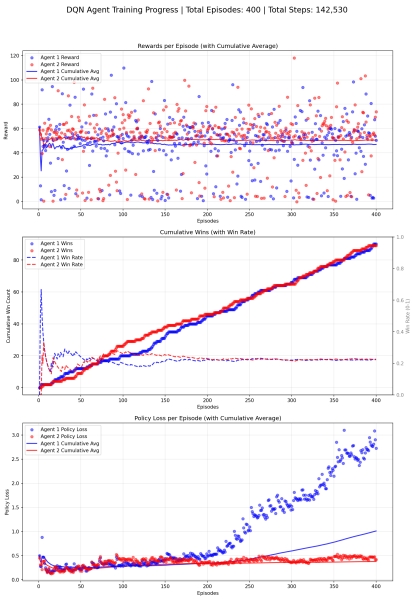

Figure B.7: Training at 400 Episodes

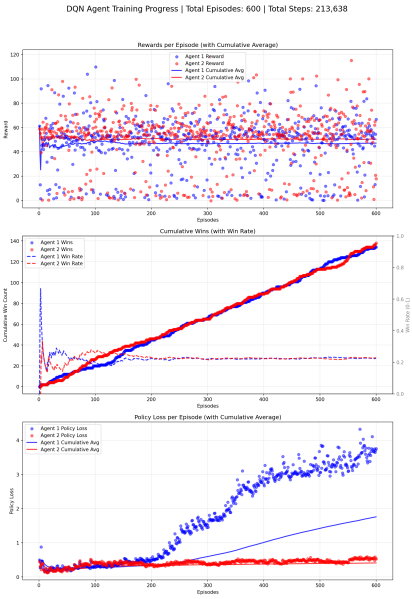

Figure B.8: Training at 600 Episodes

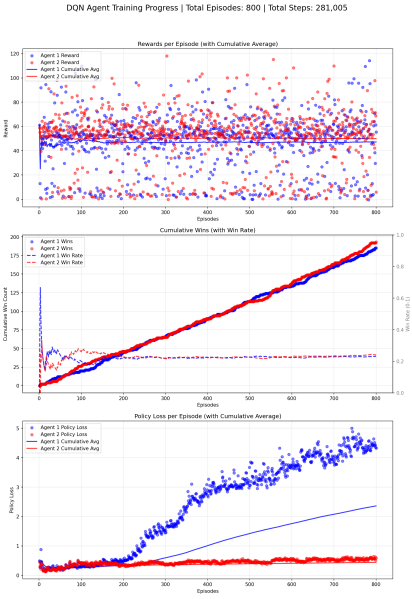

Figure B.9: Training at 800 Episodes

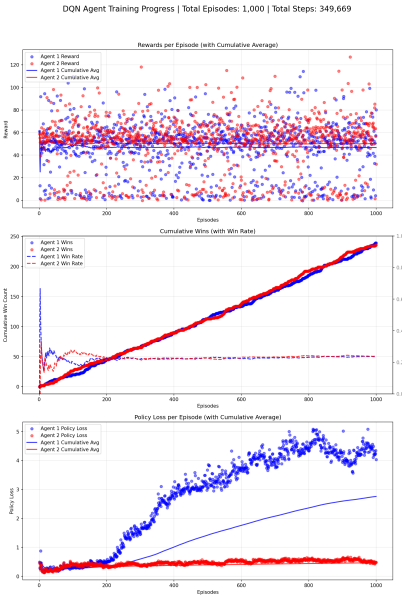

Figure B.10: Training at 1000 Episodes

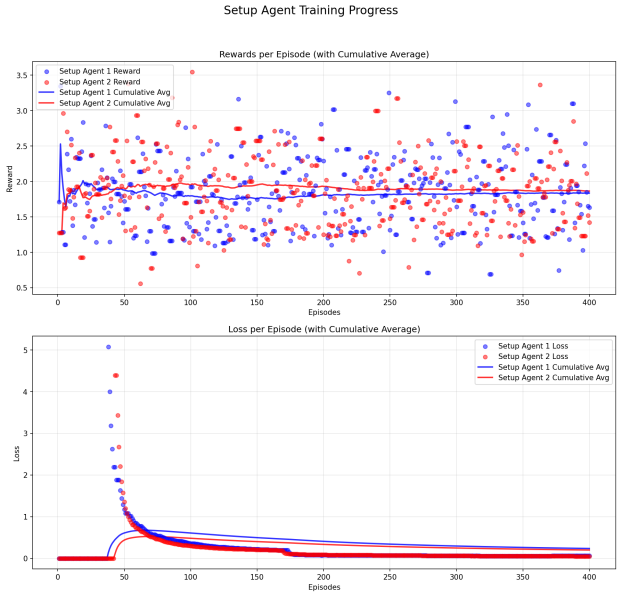

Figure B.11: Setup Agent Training

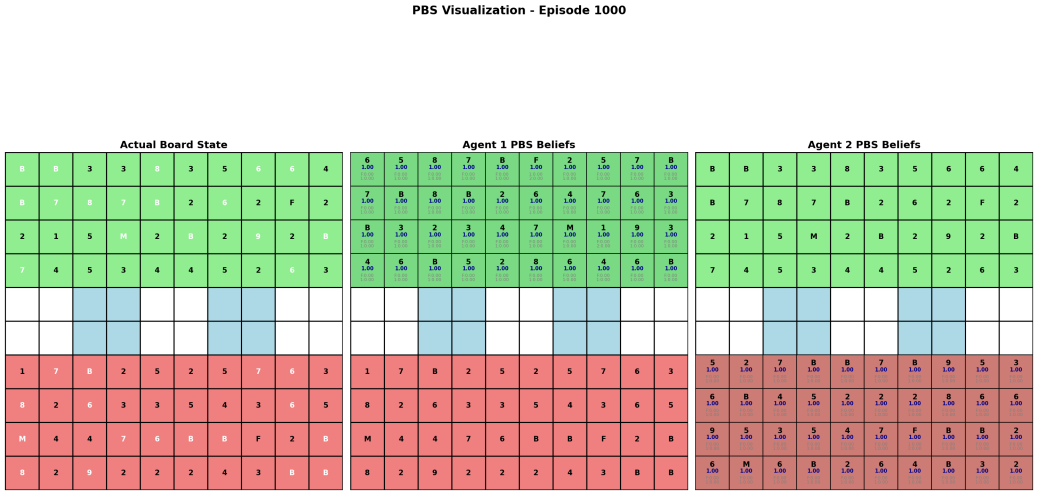

Figure B.12: PBS Visualization

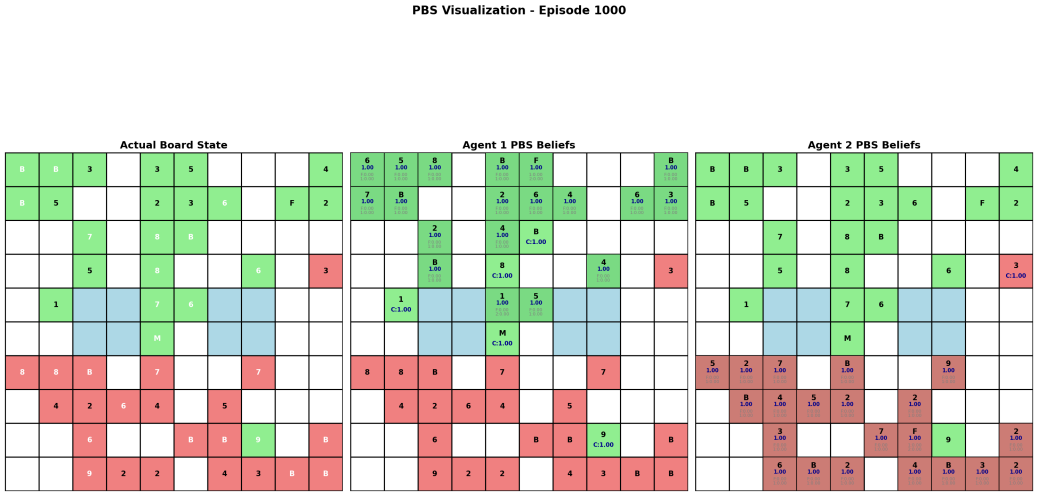

Figure B.13: PBS Visualization

REFERENCES

[1] G. Adams, S. Padmanabhan, and S. Shekhar, *Resolving implicit coordination in multi-agent deep rl with deep q-networks & game theory*, arXiv preprint arXiv:2012.09136, (2023).

[2] S. M. Al-Selwi, M. F. Hassan, S. J. Abdulkadir, A. Muneer, E. H. Sumiea, A. Alqushaibi, and M. G. Ragab, *Rnn-lstm: From applications to modeling techniques and beyond—systematic review*, Journal of King Saud University - Computer and Information Sciences, 36 (2024), p. 102068.

[3] R. N. Amaliani, D. W. Soewardikoen, H. Azhar, and Y. Rahman, *Designing board games as an enhancer of a sense of community and cognition for the elderly of tresna werdha budi pertiwi social care*, Advances in Social Humanities Research, 3 (2025).

[4] G. Barzilai, O. Shamir, and M. Zamani, *Are convex optimization curves convex?*, arXiv preprint arXiv:2503.10138, (2025).

[5] T. Becker and Z. Sunberg, *Imperfect information games and counterfactual regret minimization in space domain awareness*, in Advanced Maui Optical and Space Surveillance Technologies Conference (AMOS), AMOS Technologies, 2022.

[6] A. Brim, *Deep reinforcement learning pairs trading with a double deep q-network*, in Proceedings of the 10th Annual Computing and Communication Workshop and Conference (CCWC), Las Vegas, Nevada, USA, Jan.–2020 2020, pp. 222–227.

[7] N. Brown, A. Lerer, S. Gross, and T. Sandholm, *Deep counterfactual regret minimization*, in International conference on machine learning, PMLR, 2019, pp. 793–802.

[8] L. Canese, G. C. Cardarilli, L. Di Nunzio, R. Fazzolari, D. Giardino, M. Re, and S. Spanò, *Multi-agent reinforcement learning: A review of challenges and applications*, Applied Sciences, 11 (2021), p. 4948. Article number; MDPI journal uses article numbers instead of page ranges.

[9] P. P. Chitayat, A. M. Colman, R. J. Hyde, A. Drachen, and P. Cowling, *Wards: Modelling the worth of vision in MOBA's*, in Intelligent Systems and Applications - Proceedings of the 2020 Intelligent Systems Conference (IntelliSys), vol. 1251 of Advances in Intelligent Systems and Computing, Springer, 2020, pp. 55–76.

[10] H. M. Dbouk, K. Ghorayeb, H. Kassem, H. Hayek, R. Torrens, and O. Wells, *Facility placement layout optimization*, Journal of Petroleum Science and Engineering, 207 (2021), p. 109079.

[11] B. De Schutter, R. Babuska, and L. Busoniu, *A comprehensive survey of multi-agent reinforcement learning*, IEEE Transactions on Systems, Man, and Cybernetics, Part C: Applications and Reviews, 38 (2008), pp. 156–172.

[12] I. El Shar and D. Jiang, *Weakly coupled deep q-networks*, in Advances in Neural Information Processing Systems 36, Curran Associates, Inc., 2023. NeurIPS 2023 Conference Track.

[13] L. Feng, F. Tung, H. Hajimirsadeghi, M. O. Ahmed, Y. Bengio, and G. Mori, *Attention as an rnn*, arXiv preprint arXiv:2405.13956, (2024).

[14] J. Harrington, *Let's meetup? board game communities in hong kong*, Games and Culture, 20 (2025), pp. 279–297.

[15] C. Hu, Y. Zhao, Z. Wang, H. Du, and J. Liu, *Games for artificial intelligence research: A review and perspectives*, IEEE Transactions on Artificial Intelligence, 5 (2024), pp. 5949–5968.

[16] Y. Huang, *Deep q-networks*, in Deep Reinforcement Learning: Fundamentals, Research, and Applications, S. Z. Hao Dong, Zihan Ding, ed., Springer Nature, 2020, ch. 4, pp. 135–160. `http://www.deepreinforcementlearningbook.org`.

[17] Y. Huang, *Deep Q-Networks*, Springer Singapore, Singapore, 2020, pp. 135–160.

[18] D. Hugi and E. Izy, *Multi-agent deep reinforcement learning (marl) in starcraft 2.* `https://crml.eelabs.technion.ac.il/projects/multi-agent-deep-reinforcement-learning-marl-in-starcraft-2/`, 2017. Course project, Winter semester.

[19] A. Kaur, K. Kumar, A. Prakash, and R. Tripathi, *Imperfect csi-based resource management in cognitive iot networks: A deep recurrent reinforcement learning framework*, IEEE Transactions on Cognitive Communications and Networking, 9 (2023), pp. 1271–1281.

[20] A. Kensert, P. Libin, G. Desmet, and D. Cabooter, *Deep reinforcement learning for the direct optimization of gradient separations in liquid chromatography*, Journal of Chromatography A, 1720 (2024), p. 464768.

[21] P. Koudela, *Game of incomplete information and national security*, Central European Political Science Review (CEPSR), 20 (2019), pp. 73–96.

[22] B. Li, Z. Fang, and L. Huang, *Rl-cfr: Improving action abstraction for imperfect information extensive-form games with reinforcement learning*, arXiv preprint arXiv:2403.04344, (2024).

[23] S. Li, Y. Zhang, X. Wang, W. Xue, and B. An, *Cfr-mix: Solving imperfect information extensive-form games with combinatorial action space*, in Proceedings of the Thirtieth International Joint Conference on Artificial Intelligence (IJCAI-21), IJCAI, 2021, pp. 3663–3669.

[24] I. Loshchilov and F. Hutter, *Decoupled weight decay regularization*, arXiv preprint arXiv:1711.05101, (2017).

[25] Z. Luo, Z. Chen, and J. Welsh, *Multi-agent reinforcement learning with deep networks for diverse q-vectors*, arXiv preprint arXiv:2406.07848v1, (2024). Version v1, June 2024.

[26] R. R. Nair, T. Babu, S. Kishore, K. Pavithra, and S. G. Sumana, *Improving alpha-beta pruning efficiency in chess engines*, in 2024 International Conference on IT Innovation and Knowledge Discovery (ITIKD), Manama, Bahrain, 2025, IEEE, pp. 1–6.

[27] X. Ouyang and T. Zhou, *Imperfect-information game ai agent based on reinforcement learning using tree search and a deep neural network*, Electronics, 12 (2023), p. 2453.

[28] J. Perolat, B. De Vylder, D. Hennes, E. Tarassov, F. Strub, V. de Boer, P. Muller, J. T. Connor, N. Burch, T. Anthony, et al., *Mastering the game of stratego with model-free multiagent reinforcement learning*, Science, 378 (2022), pp. 990–996.

[29] C. Petersen, *The reality of the board game community: And the connection, identity and experience that creates it*, master's thesis, Eastern University, Aug. 2024.

[30] T. Qu, Q. Zhou, J. Zhu, and F. Duan, *Strategy optimization of imperfect information games based on nfsp with ddqn*, in Advances in Guidance, Navigation and Control, L. Yan, H. Duan, and Y. Deng, eds., vol. 845 of Lecture Notes in Electrical Engineering, Springer, Singapore, 2023, pp. 4376–4383.

[31] A. Saffidine, H. Finnsson, and M. Buro, *Alpha-beta pruning for games with simultaneous moves*, in Proceedings of the AAAI Conference on Artificial Intelligence, no. 1, 2021, pp. 556–562.

[32] M. Schmid, M. Moravčík, N. Burch, R. Kadlec, J. Davidson, K. Waugh, N. Bard, F. Timbers, M. Lanctot, G. Z. Holland, et al., *Student of games: A unified learning algorithm for both perfect and imperfect information games*, Science Advances, 9 (2023), p. eadg3256.

[33] M. Schmid, M. Moravčík, N. Burch, R. Kadlec, J. Davidson, K. Waugh, N. Bard, F. Timbers, M. Lanctot, G. Z. Holland, E. Davoodi, A. Christianson, and M. Bowling, *Student of games: A unified learning algorithm for both perfect and imperfect information games*, Science Advances, 9 (2023), p. eadg3256.

[34] M. Schofield and M. Thielscher, *General game playing with imperfect information*, Journal of Artificial Intelligence Research, 66 (2019), pp. 901–935.

[35] H. Xu, K. Li, H. Fu, Q. Fu, and J. Xing, *Autocfr: Learning to design counterfactual regret minimization algorithms*, in Proceedings of the Thirty-Sixth AAAI Conference on Artificial Intelligence (AAAI-22), Association for the Advancement of Artificial Intelligence, 2022.

[36] H. Xu, K. Li, H. Fu, Q. FU, J. Xing, and J. Cheng, *Dynamic discounted counterfactual regret minimization*, in The Twelfth International Conference on Learning Representations, 2024.

[37] A. Zakharenkov and I. Makarov, *Deep reinforcement learning with dqn vs. ppo in vizdoom*, in 2021 IEEE 21st International Symposium on Computational Intelligence and Informatics (CINTI), Budapest, Hungary, 2021, IEEE, pp. 131–136.

[38] B. Zhang and T. Sandholm, *Subgame solving without common knowledge*, Advances in Neural Information Processing Systems, 34 (2021), pp. 23993–24004.

[39] B. H. Zhang and T. Sandholm, *General search techniques without common knowledge for imperfect-information games, and application to superhuman fog of war chess*, arXiv preprint arXiv:2506.01242, (2025).

[40] K. Zhang, Z. Yang, and T. Başar, *Multi-agent reinforcement learning: A selective overview of theories and algorithms*, Handbook of reinforcement learning and control, (2021), pp. 321–384.

[41] S. Zheng, A. Trott, S. Srinivasa, N. Naik, M. Gruesbeck, D. C. Parkes, and R. Socher, *The ai economist: Improving equality and productivity with ai-driven tax policies*, arXiv preprint arXiv:2004.13332, (2020).

[42] K. Zhu, Q. Liu, X. Yan, F. Wu, X. Zhao, P. Zhou, and X. Yang, *A comprehensive survey of multi-agent reinforcement learning*, IEEE Transactions on Neural Networks and Learning Systems, 34 (2022), pp. 3074–3093.

VITA

James Gabriel P. Mabagos is BS Computer Science student of the Department of Computer Science at the Ateneo de Naga University.

Mark Lawrence M. Quibot is BS Computer Science student of the Department of Computer Science at the Ateneo de Naga University.